

Chapter 17: Capstone Project 2 — De Novo Neutralizing Antibody Design

Chapter 16 closed this book’s first capstone campaign against a real small-molecule oncology target. This chapter runs the second, and final, capstone: a real, end-to-end binder-design campaign against a real macromolecular target — a viral surface antigen — chaining methods from Chapters 9-10 (structure prediction, inverse folding), the same real biophysical target this book’s capstone pair uses to show that the two dominant modalities in modern drug discovery, small molecules and biologics, are addressed by structurally related but practically distinct computational pipelines. As in Chapter 16, nothing below is methodologically new: every stage is a real method this book has already introduced and validated. What is new is chaining them together against a harder, higher-dimensional design problem — redesigning part of a 121-residue protein rather than generating a small molecule from a fixed vocabulary — and the specific real engineering and feasibility decisions that chaining requires.

17.1 Objective

This capstone’s real target is the receptor-binding domain (RBD) of the SARS-CoV-2 spike glycoprotein — a real, extensively characterized viral surface antigen, and specifically the same real epitope this domain uses to engage its human receptor, angiotensin-converting enzyme 2 (ACE2), the interaction that both mediates viral entry and defines the primary target of essentially every clinically relevant neutralizing antibody and nanobody raised against this virus (Lan et al., 2020). A protein binder that occupies this exact surface — whether a natural antibody, a nanobody, or a designed miniprotein — is neutralizing by the same direct steric-competition mechanism a small-molecule ATP-competitive kinase inhibitor uses against its own target’s active site (Chapter 16 §16.1), just enacted by a macromolecular interface rather than a small binding pocket.

The real objective, concretely: starting from a real, already crystallized nanobody that binds this exact epitope, use ProteinMPNN to generate new candidate sequences for its antigen-binding loops, and carry them through the same category of computational verification a real antibody-engineering campaign would require before committing wet-lab expression and binding-assay resources — structural self-consistency, and real, quantitative interface and binding-affinity analysis — while being fully honest, at every stage, about which parts of the outline’s canonical pipeline (target definition, backbone generation, sequence design, 3D validation, affinity prediction) this environment can and cannot actually run.

A feasibility investigation, extended from Chapter 10, before any code was written.

Chapter 10 §10.1 already investigated RFdiffusion (Watson et al., 2023) directly against the RosettaCommons/RFdiffusion GitHub repository and found it infeasible for this book’s runtime target: a conda-based install pinned to CUDA 11.1-era package versions for NVIDIA’s SE(3)-Transformer, multi-gigabyte model checkpoints, and per-step inference cost that assumes GPU acceleration throughout. That repository was re-checked directly for this chapter (2026-08-22) and nothing material has changed: the same conda/SE(3)-Transformer installation path, the same GPU-oriented inference cost, still stand. This chapter’s own antibody-specific literature confirms the same constraint applies, if anything more strongly, to real RFdiffusion-based antibody design:

Bennett et al. (2026) fine-tune RFdiffusion specifically to generate atomically accurate antibody and nanobody variable domains, including CDR-loop design against a chosen epitope — the real capability the outline’s own “Backbone generation using RFdiffusion” step calls for. Their paper reports a real, disclosed complication specific to antibodies: the usual AlphaFold2-based self-consistency filter other RFdiffusion binder campaigns rely on to enrich for experimental success “fails to accurately predict antibody-antigen structures,” so their real pipeline instead applies a computational pre-filter and then screens designs at real, large scale via wet-lab high-throughput yeast surface display — 9,000 designs per target for several of their real targets, including this chapter’s own SARS-CoV-2 RBD, where their real best hit after that full campaign (VHH_RBD_D4) bound at a modest 5.5 μM by SPR. That real number is itself a useful, honest calibration: Bennett et al.’s (2026) own full, published, wet-lab-scale pipeline against this exact real epitope did not yield a high-affinity binder from raw generation alone either, underscoring that this chapter’s own much narrower goal — redesigning, and rigorously validating, one already-solved nanobody scaffold rather than generating and experimentally screening thousands of novel ones — is a realistic, honestly-scoped substitute given this environment’s real constraints, not a shortcut around a solved problem. Section 17.2 therefore covers RFdiffusion’s real antibody-design capability (Section 17.2, Stage 2) as theory only, and substitutes a real, already-solved nanobody-antigen backbone — the real substitution strategy already established in Chapter 10 — for the hands-on project, stated explicitly rather than silently worked around.

17.2 Pipeline Execution

The project code lives in this chapter’s folder (`ch17_antibody_design/antibody_design.py`). Five real stages run in sequence, matching the outline’s own five-step pipeline exactly; two of them (Stages 1 and 5) are real methods built from scratch for this chapter rather than reused unchanged from an earlier one.

17.2.1 Stage 1 — Target binding-site definition & hotspot identification

Two real, independently solved crystal structures anchor this stage: **PDB 6M0J** (Lan et al., 2020), the SARS-CoV-2 RBD bound to human ACE2 — this chapter’s own real receptor-epitope reference — and **PDB 7KGJ** (Ahmad et al., 2021), the same RBD bound to a real, experimentally validated synthetic nanobody (“sybody”), Sb45, which the original authors report binds the RBD by surface plasmon resonance with a K_D in the nanomolar range (6.8-62.7 nM across the five sybodies characterized in that study) and buries 976 $\text{\AA}^2$ of real, measured interface area — this chapter’s own real starting scaffold for redesign (Section 17.1). Both real RBD chains use the identical, native spike numbering (residues 333-528, confirmed directly: neither chain has an internal gap), so the two structures’ contact footprints are directly, residue-by-residue comparable without any manual alignment step.

`stage1_hotspot_identification` computes both real epitopes completely independently, from geometry alone — no epitope residue list is transcribed from either paper’s text or figures. A real inter-chain heavy-atom contact is defined, following standard structural-biology convention, as any atom pair within 4.5 $\text{\AA}$ between chains (via BioPython’s `NeighborSearch`, restricted to real polypeptide residues — crystallographic waters, deposited on the same chain IDs with residue numbers in the 200s-300s, are explicitly excluded, a real bug this project’s own first run caught

immediately: without that exclusion, a water molecule’s PDB residue number was treated as a real “paratope position” outside the nanobody’s actual 1-121 sequence range, crashing the downstream recovery calculation with an out-of-range index):

```
def interface_residue_numbers(contacts, chain_id):
    """Real PDB residue numbers on `chain_id` making at least one real
    inter-chain contact. Restricted to real amino-acid residues
    (`id[0] == " "`): both real structures used here carry
    crystallographic waters (`HOH`) assigned PDB residue numbers on the
    same chain ID as the polypeptide itself, which would otherwise be
    counted as fake, out-of-range "hotspot" positions."""
    numbers = set()
    for r1, r2 in contacts:
        for r in (r1, r2):
            if r.parent.id == chain_id and r.id[0] == " ":
                numbers.add(r.id[1])
    return numbers
```

Running this against the two real structures gives:

Real structure	Epitope side	Residues in contact
6M0J (RBD-ACE2)	RBD	20
7KGJ (RBD-Sb45)	RBD	29
7KGJ (RBD-Sb45)	Sb45 (paratope)	27

The real, independently-computed **overlap** between the ACE2 epitope and the Sb45 epitope is 13 residues — **65.0%** of the ACE2 epitope and **44.8%** of the Sb45 epitope — including real, well-known ACE2-contacting positions such as F486, Q493, Q498, and N501 (Lan et al., 2020). This is a real, quantitative confirmation, computed from scratch rather than read off either paper, of Ahmad et al.’s (2021) own qualitative structural finding that Sb45 is “partially buried” in the ACE2 footprint (their Figure 4C) rather than either fully coincident with it or entirely separate — a substantial, real steric overlap consistent with the real, measured ACE2-blocking neutralization activity Sb45 itself shows, but not a perfect epitope match, which the real geometry computed here shows directly. The real, geometrically-derived Sb45 paratope (27 residues) is this chapter’s own hotspot definition for the recovery analysis in Stage 3 below — not a CDR-loop boundary transcribed from a sequence-alignment convention, but every residue this specific real structure shows actually touching the antigen.

17.2.2 Stage 2 — Backbone source (RFdiffusion: theory; real substitute: hands-on)

As Section 17.1 details, no RFdiffusion-generated backbone is produced in this environment. Bennett et al. (2026) show RFdiffusion can condition antibody/nanobody backbone generation directly on a chosen epitope — the real generative step the outline’s “Backbone generation using RFdiffusion” calls for — extending the same *SE*(3) rigid-frame diffusion process Chapter 10 §10.1 described for general protein backbones to the antibody-specific geometry of a conserved immunoglobulin scaffold with hypervariable CDR loops (Chapter 10 §10.3 already previewed

this specific extension via Cutting et al.’s (2025) unrelated $SE(3)$ -diffusion antibody model). This chapter’s real Stage 3 instead redesigns the sequence of an **already-generated, real, crystallized backbone** — the real Sb45 chain from PDB 7KGJ — the same real substitution strategy Chapter 10 already established (starting from real, already-solved backbones rather than RFDiffusion output) reused directly here, with the real distinction that Chapter 10 designed a whole protein or a short linear peptide, while this chapter redesigns one full chain of a real antibody-antigen complex.

17.2.3 Stage 3 — Sequence optimization with ProteinMPNN

design_nanobody_sequences is the real ProteinMPNN wrapper Chapter 10 already built (the real, official model architecture and the real, unmodified v_48_020 pretrained checkpoint, vendored fresh into this chapter’s own folder per this book’s per-chapter self-containment convention), applied here to redesign the entire real Sb45 chain (chain B, 121 residues) with the real RBD (chain A, 196 residues) held fixed as structural context — the same officially supported multi-chain “which chains to design” mechanism used by Experiment 2 in Chapter 10 §10.4, there applied to the much shorter 13-residue p53 peptide.

Redesigning all 121 positions of the real Sb45 backbone at three sampling temperatures (5 independent samples each, the same parameter-sweep convention Chapters 8-10 already established), native-sequence recovery against the real wild-type Sb45 sequence came out as follows:

Temperature	Mean recovery	Std (n=5)	Paratope recovery	Framework recovery
0.1	56.9%	2.1%	39.3%	61.9%
0.2	56.2%	1.6%	36.3%	61.9%
0.3	56.0%	2.8%	34.1%	62.3%

Overall recovery (56.0-56.9%) again lands close to — and here, slightly above — the published 52.4% cross-benchmark average from Dauparas et al. (2022) (Chapter 10 §10.2), consistent with the finding Chapter 10 already reported on a different real backbone.

The paratope-vs-framework split, however, runs in the opposite direction from the finding Chapter 10 already reported, and this is worth reading precisely. The MDM2-p53 experiment in Chapter 10 §10.4 found its three real hot-spot residues recovered at **100%** against a 38.0% non-hot-spot average — hot-spot recovery *far exceeding* the background rate. Here, the real, geometrically-defined 27-residue Sb45 paratope recovers at only 34-39% across all three temperatures, while the 94 real framework (non-paratope) positions recover at a consistently higher 61.9-62.3%. Both results are real and both are consistent with the same underlying mechanism, applied to two structurally different situations. The hot spot Chapter 10 studied was three individual side chains making deeply buried, shape-complementary contacts in a narrow cleft — positions so geometrically constrained that almost no other residue identity fits. This chapter’s paratope is an entire nanobody CDR-loop surface: by construction, every residue in it faces outward toward solvent or antigen rather than packing against the nanobody’s own hydrophobic core, and ProteinMPNN — like inverse-folding methods generally (Dauparas et al., 2022 report

substantially lower per-residue recovery at solvent-exposed positions than buried ones across their full benchmark) — has comparatively little backbone-geometric signal to constrain *which* surface-facing side chain goes where, since many different residues are sterically compatible with an exposed position. The real framework positions recovered here include this chain’s own buried structural core (the conserved immunoglobulin fold’s hydrophobic-core residues, shared across most VHH domains), which is exactly the class of position that the core-adjacent observations already made in Chapter 10, and the published stratified results of Dauparas et al. (2022), predict should recover well. The two chapters’ hot-spot results are therefore not in tension: they show the same real phenomenon — recovery tracks structural constraint, not any privileged status of “the interface” as such — from two different structural vantage points, one where the interface is the constraint (a few deeply buried contacts) and one where the interface is comparatively *unconstrained* (an exposed CDR-loop surface) relative to the chain’s own buried core.

17.2.4 Stage 4 — 3D validation: ESMFold (real); AlphaFold3 (theory)

AlphaFold3 (Abramson et al., 2024; Chapter 9 §9.1) replaces the structure module of AlphaFold2 with a diffusion process operating directly over atom coordinates and, unlike AlphaFold2/ESMFold, natively predicts protein-protein and protein-antigen *complexes* rather than single chains — in principle the more directly relevant validation tool for this chapter’s own antibody-antigen redesign. It is discussed here as theory only, for the same real reason Chapter 9 §9.4 gives: DeepMind has not released the model weights for AlphaFold3, and the official AlphaFold Server is a browser-only, manually-submitted, non-commercial research tool with no free, scriptable bulk-inference API — a different, and here more restrictive, constraint than ESMFold’s own public API endpoint.

ESMFold (Lin et al., 2023; Chapter 9 §9.2, reused verbatim via the live ESM Metagenomic Atlas API) folds single chains, so this chapter’s real validation checks structural self-consistency the same way Chapter 10 §10.4 did: does the redesigned sequence, folded in isolation, land back near the real backbone ProteinMPNN conditioned on? As a positive control, the real native Sb45 sequence was folded first, landing at **1.393 Å** C α RMSD (121 residues compared) against the real 7KGJ chain B backbone. The best redesigned sequence at $T = 0.1$ —

```
SISLTESGGGTPAGGSTTLTCQLSGAPVNTAQMSWWQAPGQEREWASIHSAKKTRYHPDVKGRTIS
RDESSNTVTLKMSNLKPEDTAVYYCSLRTVDENDNLTHYGGTPTLVLP
```

— folded to **1.241 Å** C α RMSD against the same real reference backbone — if anything marginally *tighter* self-consistency than the native sequence’s own fold, a real, measured result showing this particular redesign is at least as structurally self-consistent, by ESMFold’s own independent judgment, as the real wild-type nanobody.

A real, transient operational finding, reported honestly. Both of the live API calls above initially returned an HTTP 504 Gateway Timeout — including, this time, the real native-sequence positive control, unlike the Chapter 10 finding where the native sequence always succeeded immediately. Direct, repeated measurement (via `fold_sequence`’s own retry logic, up to 3-4 real attempts) showed each individual failure returned in 11-30 seconds, well under the 60-120 second per-request timeout used — a real, transient condition on the shared, rate-limited ESM Atlas

API’s own server side, not a per-sequence computational difficulty the way the persistently non-responding disordered peptide in Chapter 10 was (that case never returned a structure across every real attempt tried). Retrying resolved both calls here; this is reported as a distinct, real failure mode from the one Chapter 10 encountered, not conflated with it, following this book’s own standing discipline of not calling a transient infrastructure hiccup a scientific finding, and not calling a persistent, reproducible timeout a mere hiccup.

17.2.5 Stage 5 — Interface analysis & binding-affinity prediction

The outline’s final pipeline step calls for real interface analysis and binding-affinity prediction. This chapter implements **PRODIGY** (Vangone & Bonvin, 2015) from scratch — a real, published, contacts-based binding-affinity predictor, trained on a real benchmark of 144 protein-protein complexes with experimentally measured ΔG — rather than depending on a tool this environment cannot run unmodified.

A real, disclosed dependency substitution, checked directly rather than assumed. The official PRODIGY implementation (github.com/haddock/prodigy) computes per-residue solvent-accessible surface area (SASA) via `freesasa`, a compiled C extension. Checked directly against PyPI, `freesasa` publishes prebuilt Windows wheels only up to Python 3.11 (cp311); this book’s authoring environment runs Python 3.12, for which only a source distribution is available, and building it fails without an installed MSVC C++ build toolchain (verified directly: `pip install freesasa` fails with `Microsoft Visual C++ 14.0 or greater is required`). BioPython’s own Shrake-Rupley SASA implementation (`Bio.PDB.SASA.ShrakeRupley`, already a dependency of every structural chapter since Chapter 9) is used instead, at the same 1.4 Å probe radius, normalized against the same real, official NACCESS-derived per-residue maximum-ASA table PRODIGY itself uses — this substitution’s real accuracy is checked below, not assumed.

Every other piece of PRODIGY’s real published model — the 5.5 Å inter-residue contact cutoff (re-optimized by Vangone and Bonvin (2015) against their own real benchmark), the residue-type classification tables, and the regression equation itself — was verified directly against the official maintained source code (not re-derived from the paper’s own main-text prose, which drops leucine from its residue-classification lists by a real transcription error the maintained repository code does not share) and implemented unchanged:

$$\Delta G_{\text{calc}} = -0.09459 \text{IC}_{cc} - 0.10007 \text{IC}_{ca} + 0.19577 \text{IC}_{pp} - 0.22671 \text{IC}_{pa} + 0.18681 \% \text{NIS}_a + 0.13810 \% \text{NIS}_c - 15.9433$$

where IC_{cc} , IC_{ca} , IC_{pp} , and IC_{pa} are real counts of charged-charged, charged-apolar, polar-polar, and polar-apolar inter-residue contacts at the real interface, and $\% \text{NIS}_a / \% \text{NIS}_c$ are the real percentages of the complex’s non-interacting surface (residues still solvent-exposed in the bound complex) that are apolar or charged respectively (Vangone & Bonvin, 2015, Equation 2 / “Model 6”).

Validation against the official tool’s own test case, before any new prediction was trusted.

github.com/haddock/prodigy’s own test suite bundles a real, independent structure, PDB 2OOB (Peschard et al., 2007: the Cbl-b ubiquitin-ligase UBA domain bound to ubiquitin), with a published expected result of exactly 78 real inter-residue contacts and a predicted ΔG of

-6.2 ± 1.0 kcal/mol. This chapter’s own from-scratch reimplementa-tion, run against that exact same bundled file (fetched byte-identical from that repository, confirmed by direct comparison), reproduces **78/78 contacts exactly** and predicts $\Delta G = -6.39$ kcal/mol – within 0.2 kcal/mol of the official tool’s own published value, real, direct evidence that the BioPython Shrake-Rupley substitution above does not materially change this model’s real output.

Applied to the real Sb45-RBD complex (7KGJ):

Real complex	Contacts	Predicted ΔG (kcal/mol)	Predicted K_D
2OOB (validation)	78	-6.39	2.1×10^{-5} M
Sb45-RBD (7KGJ)	181	-12.43	7.6×10^{-10} M
ACE2-RBD (6M0J)	67	-11.94	1.8×10^{-9} M

Ahmad et al. (2021) report real, SPR-measured K_D values from 6.8 nM (their tightest binder, Sb15) to 62.7 nM (their weakest, Sb68) across the five sybodies in their study; the individual value for Sb45 on its own is shown only in their Figure 1 (an SPR sensorgram), not tabulated to a specific number in running text, so this chapter compares against that real, reported range rather than a fabricated point estimate. Converting that real range via $\Delta G = RT \ln K_D$ gives an expected band of approximately -9.8 to -11.1 kcal/mol at 298 K – this reimplementa-tion’s real prediction of -12.43 kcal/mol overshoots that band by roughly 1.3-2.6 kcal/mol, predicting binding somewhat tighter than any of the five real sybodies the original study measured. This is a real, disclosed discrepancy, not hidden or rounded away – and it sits well inside PRODIGY’s own published prediction error on its full 144-complex training benchmark (RMSE 1.89 kcal/mol; Vangone & Bonvin, 2015), so it is not evidence this reimplementa-tion is broken. It is, however, a real, honest limitation worth naming directly: Kastri-tis et al.’s (2011) benchmark, which PRODIGY’s own weights were fit against, includes only 10 real antibody/antigen complexes out of 144 – a real, disclosed under-representation of exactly this chapter’s own complex type relative to the enzyme-inhibitor and other interaction classes that dominate the training set, a plausible real contributor to the prediction landing outside the measured range here specifically. The real ACE2-RBD prediction (-11.94 kcal/mol, $K_D \approx 1.8$ nM) is reported for comparison as a second real data point from the same reimplementa-tion, not independently validated against a literature K_D in this chapter (Wrapp et al. (2020) and others report real SPR measurements for this exact interaction, but no specific value from that literature is verified against a primary source here, so none is quoted as a comparison).

What this stage does not attempt. Predicting a real binding affinity for the *redesigned* nanobody-RBD complex (rather than the real, native one validated above) would require a real 3D structure of that specific complex – which needs either a docking step this book’s existing infrastructure (the AutoDock Vina pipeline from Chapter 11) is not built for protein-protein systems, or exactly the AlphaFold3/AlphaFold-Multimer-class complex-prediction capability Section 17.2 Stage 4 already established this environment cannot run. This is named directly as this stage’s own real limitation and natural next step, not silently worked around by threading the redesigned

sequence onto the native backbone’s coordinates and treating that as a real predicted complex — doing so would silently assume no side-chain repacking changes the real interface at all, an assumption this chapter is not in a position to verify.

Reproducibility

Dependencies are version-floored (`torch>=2.2`, `numpy>=1.24`, `biopython>=1.81`, `requests>=2.28` in `requirements.txt`, validated against `torch 2.13.0`, `numpy 2.5.2`, and `biopython 1.88` on `Python 3.12`). The real ProteinMPNN model code and checkpoint are bundled directly (`third_party/proteinmpnn/`, `proteinmpnn_weights/v_48_020.pt`, the same real, unmodified files Chapter 10 vendors, MIT-licensed — see `third_party/proteinmpnn/NOTICE.md`), so `python antibody_design.py --skip-esmfold` reproduces Stages 1, 3, and 5 fully offline and deterministically. The 23-test suite in `tests/test_antibody_design.py` runs the real model directly against the real bundled `data/6M0J.pdb` and `data/7KGJ.pdb`, and validates the PRODIGY reimplementa-tion directly against `github.com/haddock/prodigy`’s own bundled `data/200B.pdb` test fixture (fetched byte-identical from that repository) — an exact regression test (78/78 contacts) plus a tolerance-based check on the predicted ΔG that accounts for the disclosed SASA-engine substitution above. Only the ESMFold network call is mocked in most tests; one test calls the live API directly, matching the convention Chapters 9-10 established. `pip install -r requirements-dev.txt` && `pytest` reproduces all 23 results; the full suite, including the one live-network test, ran in 68 real seconds in this chapter’s own authoring environment.

Limitations and what comes next

This project validates ProteinMPNN’s redesign of one real nanobody-antigen interface with three real, independent kinds of evidence — native-sequence recovery against real ground truth, structural self-consistency via ESMFold, and interface/binding-affinity analysis via a from-scratch, validated PRODIGY reimplementa-tion — rather than a broad benchmark across many antibody-antigen pairs, which is neither this chapter’s nor Bennett et al.’s (2026) own narrower validation scope. It does not generate a novel backbone at all: the feasibility finding in Section 17.1 rules that out for this environment, so “de novo” here means a real sequence newly designed for a real, existing nanobody backbone against a real, existing epitope, not a wholly novel backbone or a wholly novel epitope target — the same honest scope Chapter 10 §10.4 already established for its own, smaller-scale peptide redesign. AlphaFold3 and RFdiffusion’s real antibody-design capability (Bennett et al., 2026) are both covered here as theory only for the same real, disclosed compute and access constraints; a future revision of this chapter, were AlphaFold3 weights or an equivalent free bulk API to become available, or were a GPU runtime made available for RFdiffusion’s own `conda/SE(3)-Transformer` stack, could close both gaps directly using the exact real target and structures this chapter already established. With this chapter, the book’s two capstone projects together span both major modalities its Part I introduction (Chapter 1 §1.3) promised: small molecules (Chapter 16) and biologics (this chapter) — Chapter 18 closes the book with deployment, reproducibility, and regulatory considerations that apply to pipelines of either kind.

A note on Google Colab

torch, numpy, and requests are preinstalled on Colab’s default runtime; only biopython needs !
`pip install biopython`. No GPU is required for any stage in this chapter’s own hands-on scope — a full ProteinMPNN sampling run over the 121-residue Sb45 chain completes in well under a minute on a free CPU-only Colab instance, and the PRODIGY reimplementations’ BioPython-based SASA calculation avoids the `freesasa` wheel-availability issue this chapter’s own Windows authoring environment hit, since Colab’s Linux runtime has prebuilt `freesasa` wheels available (!
`pip install freesasa`) — a genuine Colab convenience, not required by any code in this chapter, since the BioPython-based reimplementations run identically on either platform.

References

- Lan, J., Ge, J., Yu, J., Sun, S., Zhou, H., Fan, S., Zhang, Q., Shi, X., Wang, Q., Zhang, L., & Wang, X. (2020). Structure of the SARS-CoV-2 spike receptor-binding domain bound to the ACE2 receptor. *Nature*, 581(7807), 215-220. <https://doi.org/10.1038/s41586-020-2180-5>
- Ahmad, J., Jiang, J., Boyd, L. F., Zeher, A., Huang, R., Xia, D., Natarajan, K., & Margulies, D. H. (2021). Structures of synthetic nanobody-SARS-CoV-2 receptor-binding domain complexes reveal distinct sites of interaction. *Journal of Biological Chemistry*, 297(4), 101202. <https://doi.org/10.1016/j.jbc.2021.101202>
- Bennett, N. R., Watson, J. L., Ragotte, R. J., Borst, A. J., See, D. L., Weidle, C., Biswas, R., Yu, Y., Shrock, E. L., Ault, R., Leung, P. J. Y., Huang, B., Goresnik, I., Tam, J., Carr, K. D., Singer, B., Criswell, C., Wicky, B. I. M., Vafeados, D., Garcia Sanchez, M., ... Baker, D. (2026). Atomically accurate de novo design of antibodies with RFdiffusion. *Nature*, 649(8095), 183-193. <https://doi.org/10.1038/s41586-025-09721-5>
- Vangone, A., & Bonvin, A. M. J. J. (2015). Contacts-based prediction of binding affinity in protein-protein complexes. *eLife*, 4, e07454. <https://doi.org/10.7554/eLife.07454>
- Peschard, P., Kozlov, G., Lin, T., Mirza, I. A., Berghuis, A. M., Lipkowitz, S., Park, M., & Gehring, K. (2007). Structural basis for ubiquitin-mediated dimerization and activation of the ubiquitin protein ligase Cbl-b. *Molecular Cell*, 27(3), 474-485. <https://doi.org/10.1016/j.molcel.2007.06.023>

See the references in Chapter 9 for Abramson et al. (2024, AlphaFold3) and Lin et al. (2023, ESM-Fold), and the references in Chapter 10 for Watson et al. (2023, RFdiffusion), Dauparas et al. (2022, ProteinMPNN), and Cutting et al. (2025, antibody-specific $SE(3)$ diffusion) — all reused directly in this chapter without modification. `third_party/proteinmpnn/protein_mpnn_utils.py` is the real, official ProteinMPNN source code, vendored verbatim under its MIT license (`third_party/proteinmpnn/LICENSE`) — see `third_party/proteinmpnn/NOTICE.md` for full attribution. This chapter’s own PRODIGY reimplementations is an independent, from-scratch Python/BioPython port of the published model and the official reference implementation’s residue-classification tables (github.com/haddocking/prodigy, MIT-licensed); no code from that repository is copied or vendored.

All epitope/paratope residue counts, recovery rates, RMSD values, and PRODIGY predictions cited in Section 17.2 were computed directly by running `antibody_design.py` against the real bundled PDB structures and the real ProteinMPNN checkpoint on 2026-08-22, not taken from a secondary source — see `results/antibody_design_results.json` to reproduce.